CIP Protocol: Deterministic Arithmetic Operator for File Integrity

Alvaro Costa

February 16, 2026

Abstract

This work presents the CIP protocol (Primal Integrity Cipher), implemented as a `C++` binary that performs deterministic integrity verification, reversible confidentiality, and spectral verification over files. The system models a finite arithmetic operator that projects data blocks onto spectral matrices constructed from eigenvectors of a *Gaussian Orthogonal Ensemble* (GOE) matrix. Integrity is evaluated through the stable repetition of the spectral projection when the artefact remains unaltered. Encryption is implemented as a deterministic transformation based on the same projection structure. The protocol does not use external sources of randomness, seeds, or hidden state. All outputs are exclusive functions of the input and fixed internal structures.

The article describes the architecture of the binary, the operational regime, the artefacts produced, and the formal limits of the system. It is demonstrated that statistical structures of spectral universality can be used as a basis for deterministic integrity operators. The proposed architecture allows such an operator to be deployed under governance regimes compatible with principles of public access, verifiability, and non-exclusionary use.

Operational notebook available at:
https://colab.research.google.com/drive/1vmYMYuNgryfgKzC09yHnZX36Erj35i8U?usp=sharing

Any interested party, including public or private administrators, may execute the notebook and experimentally verify the behaviour described.

1 Operator Definition

The CIP binary (Primal Integrity Cipher) is an executable written in `C++`.

It implements a finite arithmetic operator.

This operator measures spectral coherence against a fixed reference ruler in a deterministic and reproducible manner.

The operator receives as input a finite binary stream and produces a set of deterministic artefacts.

There are no external sources of randomness. There are no seeds. There is no hidden state.

Every output is an exclusive function of the input and fixed internal structures.

Formally, let B be a finite vector of bytes. Define:

$$\mathcal{CIP}(B) \to \{A, K, L\}$$

Where:

- A is the audit.
- K is the key.
- L is the operational log.

The operator is closed.

The same input, under the same operational mode and the same binary version, produces exactly the same outputs.

Version changes do not characterise a new operator.

Distinct versions must preserve functional invariance.

There are no probabilistic branches. There are no stochastic choices.

The operator is defined independently of any specific implementation.

1.1 Operational definition

An SDK (Software Development Kit) is an operational package containing:

- the compiled CIP binary;
- the associated spectral matrices;
- auxiliary files required for execution;
- a standardised directory structure.

That is:

> An SDK is an operational instance of the CIP operator.

1.2 Deterministic nature

The operator is strictly deterministic.

There is no dependence on time, environment, or clock.

The execution of $\mathcal{CIP}$ is the direct application of fixed rules.

1.3 Spectral ruler

The operator uses a discrete spectral ruler.

The ruler is an ordered set of eigenvectors obtained from the spectral decomposition of a GOE-class matrix and used as a projection basis.

The ruler is unique and invariant within the protocol.

The representation of the ruler is obfuscated by design to protect confidentiality.

The definition of the ruler is public. The anchor point used by each SDK is private.

Each SDK anchors at its own point on the ruler, selected within an operational domain starting from 5×10^6.

Spectral matrices are computed from this anchor point and shipped with the SDK.

The anchor point is not exposed to the user.

Distinct SDKs may anchor at distinct points on the ruler.

1.4 Precomputed matrices

The binary ships with precomputed spectral matrices.

Signature matrices are provided with dimensions:

- 128
- 256
- 512
- 1024

An additional matrix, extracted from another point on the ruler, of dimension 128, is used for encryption.

Matrices are selected automatically according to file size.

1.5 Admissible matrix class

The operator is not compatible with arbitrary matrices.

The projection basis must belong to a class exhibiting spectral universality, statistical rigidity, and global eigenvector stability.

The projection basis used by the operator consists of a fixed subset of 16 eigenvectors extracted from the spectral decomposition of the anchored GOE matrix.

This number is constant in the protocol.

It defines the projection matrix dimension and is independent of file size.

Generic matrices, simple structured matrices (identity, diagonal, periodic blocks), or ensembles without controlled universality do not exhibit these properties and lead to projection degeneration.

The GOE class provides eigenvectors whose statistical distribution is stable, universal, and insensitive to small construction variations.

This property is required so that small perturbations in the artefact produce global displacements in the spectral projection, guaranteeing sensitivity, reproducibility, and known absence of structural collisions.

No practical adversarial mechanisms for constructing collisions under this projection class are currently known in the literature.

Therefore, the choice of the GOE class is not aesthetic. It is an operational requirement.

Without this class of properties, the operator ceases to be operational.

1.6 Ruler uniqueness and interoperability

Interoperability requires a unique ruler.

Binaries may possess their own projection bases.

Projection bases may be exclusive.

Exclusive bases guarantee interoperability only if the ruler is unique.

Exclusive bases enable traceability.

Communication between binaries occurs only when they share the same ruler.

The computational complexity of the projection is linear in the number of processed blocks.

There are no superlinear growth stages in the operator's main flow.

1.7 Projection mechanism

The file is divided into blocks.

Block size equals the dimension of the selected matrix.

Each block is projected onto the corresponding matrix.

The projection generates a spectral vector.

The spectral vector is transformed into a SHA-256 hash.

The final signature is the concatenation of the SHA-256 hashes generated for all blocks.

This result is used in artefact construction.

The key carries the representation of the projection basis used.

1.8 Coherence with unity

If the artefact remains intact, the spectral projection repeats.

The repetition of the same projection confirms coherence.

Integrity reduces to verifying this coherence.

1.9 Functional independence

The operator does not depend on human intent to function.

Once invoked, processing follows the internal protocol in full.

The operator does not interpret intention, does not adjust behaviour, and does not learn.

1.10 Execution dependence

The operator depends on human intent to be executed.

There is no spontaneous execution and no internal scheduling.

Operation occurs only through explicit binary invocation.

1.11 Domain of operation

The operator acts exclusively on files, regardless of format.

It does not operate on real-time traffic, network sessions, or communication protocols.

The domain is strictly offline.

1.12 Instrumental functions

The operator manifests three functions as projections of the same mechanism:

- Deterministic integrity
- Reversible confidentiality
- Spectral verification

There are no independent modules.

There are no decoupled subsystems.

1.13 Formal limits

The operator does not attempt to prove external mathematical properties, infer semantic meaning, classify content, or interpret data.

The operator only transforms, measures coherence with unity, and verifies.

1.14 Nature of the operator

The CIP binary is not an application, nor a service, nor a software product in the conventional sense.

It implements a mathematical operator.

The operator exists independently of specific instances, compilations, or distributors.

The binary is merely an executable materialisation of this definition.

No entity can appropriate the operator.

No government, company, or individual can make it exclusive.

The operator may be regarded as a public good of mathematical nature.

Its existence follows from the structure it describes, not from external authorisation.

The protocol reproduces the same asymmetry present in mathematics itself:

There exists a regime of stewardship of the ruler.

There exists a regime of use of the ruler.

These regimes are distinct and complementary.

Those who steward do not control those who use.

Those who use do not define the ruler.

System stability arises precisely from this separation.

1.15 Protocol identity

The CIP binary implements a protocol.

The protocol is defined by a fixed set of verbs:

- `cip assinar`
- `cip verificar`
- `cip cifrar`
- `cip decifrar`

These verbs are part of the structural definition of the protocol.

They are not natural-language interface elements.

They are not translation objects.

They are not subject to linguistic versioning.

Language changes apply only to comments, documentation, and explanatory material.

The literal preservation of the verbs guarantees interoperability, reproducibility, and historical continuity of the operator.

Any deviation from this identity breaks the protocol.

2 Operational Regime

The CIP binary operates through four fundamental verbs.

Each verb corresponds to a specific operational instance of the same arithmetic operator.

The available verbs are:

- Assinar

- Verificar
- Cifrar
- Decifrar

All verbs use the same spectral ruler and the same projection mechanism.

Differences between verbs concern only the type of matrix employed and the artefacts produced.

2.1 Modes of operation

The binary supports two operational modes: *Standard* and *Total*.

The **Standard Mode** prioritises processing efficiency and immediate integrity, generating only the key and the operational screen log.

The **Total Mode** performs an exhaustive verification of all blocks and generates a serialised audit artefact.

The mathematical mechanism remains identical.

2.2 Assinar verb

The `assinar` verb computes the deterministic signature of a file.

Matrices whose dimension is automatically selected according to file size are used.

The signature is constructed according to the projection mechanism described in Section 1.

The verb produces:

- A key (`.json`).
- An operational log (terminal output).
- An audit (`.json`), **only when invoked in Total Mode**.

The file content is not modified.

2.3 Verificar verb

The `verificar` verb validates coherence between a file and a key.

The same matrices selected according to file size are used.

The operator recalculates the file signature.

The recalculated signature is built by the same deterministic mechanism.

The result is compared with the value stored in the key.

If the values coincide, integrity is confirmed.

If the values diverge, integrity is rejected.

2.4 Cifrar verb

The `cifrar` verb produces an encrypted version of the file.

Encryption uses a single matrix of dimension 128.

One hundred and twenty-eight blocks of the file are deterministically selected according to a fixed internal rule of the protocol.

Each selected block is projected onto the matrix.

The projection output feeds an XOR stream.

The XOR stream is derived from keying material (IKM) processed by HKDF.

This stream deterministically scrambles the original file content.

The verb produces:

- Encrypted file.
- Key (`.json`).
- An operational log (terminal output).
- An audit (`.json`), **only when invoked in Total Mode**.

2.5 Decifrar verb

The `decifrar` verb reconstructs the original file from the encrypted file and the key.

The same matrix of dimension 128 is used.

The key provides the exact sequence of blocks used.

The key provides the representation of the projection basis used.

The same XOR stream is reconstructed.

The operation is reversible.

The result is a file that is binary-identical to the original, if no corruption is present.

2.6 Relationship between verbs

The verbs are not independent of one another.

All are projections of the same operator.

Differences between verbs correspond only to the type of artefact produced.

2.7 Standard mode

In standard mode, the binary generates essential artefacts.

Verification is performed on a fixed subset of file blocks.

Fifteen blocks are verified: five at the beginning, five in the middle, and five at the end.

This mode prioritises performance while maintaining corruption detection capability and support for reversible confidentiality.

2.8 Total mode

In total mode, the binary generates expanded audit data.

Verification is performed over all blocks of the file.

Additional metadata are incorporated into the key and the audit.

The mathematical mechanism remains unchanged.

Additional metadata are incorporated into the key and the audit without altering signature construction.

3 Key Structure

The key is a serialised artefact generated by the CIP binary.

The key fully describes the parameters of the executed operation.

The key is required for verification, decryption, and audit consistency.

The key does not contain the file content.

The key contains only metadata and material derived from the projection.

No field in the key allows reconstruction of the content without the corresponding file.

3.1 Mandatory fields

The key contains, at minimum, the following fields:

- Identifier of the verb used.
- Mode of operation.
- Representation of the projection basis used.
- Matrix dimension.
- Final signature (concatenated hash).
- Block sequence used in encryption, when applicable.
- Technical metadata (binary version, timestamp, file size).

All fields are serialised deterministically.

3.2 Immutability

Once generated, the key is not modified.

Any alteration to any field invalidates the key.

3.3 Relationship with the file

The key is mathematically bound to the file.

A single key does not validate distinct files.

Distinct files produce distinct keys.

3.4 Use of the key

The key is used as input to the `verificar` and `decifrar` verbs.

Absence of the key prevents these operations.

3.5 Exposure

The key may be stored, copied, and transported freely.

Possession of the key does not grant access to the encrypted file content without the file itself.

Possession of the encrypted file does not grant access to its content without the key.

4 Audit Structure

The audit is a serialised artefact generated exclusively in **Total Mode**.

In *Standard Mode*, trust resides in the key's signature and the binary's immutability, dispensing with the audit record for infrastructural performance gains.

When generated, the audit records detailed technical information about the operation's execution for inspection, traceability, and forensic analysis.

The audit is not required for verification or decryption operations. Its absence does not prevent the mathematical validation of spectral coherence, which is self-sufficiently guaranteed by the key.

4.1 Audit types in Total Mode

The `assinar` verb generates `auditoria_assinado`.

The `verificar` verb generates `auditoria_verificado`.

The `cifrar` verb generates `auditoria_cifrado`.

The file is signed before the encryption operation.

The `decifrar` verb generates `auditoria_decifrado`.

The file is verified after the decryption operation.

Audits are independent of one another.

One audit does not depend on the existence of another.

4.2 Audit contents

The audit contains technical fields that mirror the operation's parameters, including:

- `operacao`: Verb identifier and protocol version.
- `assinatura_integridade_arquivo_original`: The spectral hash of the raw data.
- `assinatura_integridade_chave_cip`: The protection seal of the key itself (signed by the guardian matrix).
- `status_autovalidacao_chave`: Result of the key's internal integrity check.
- `matriz_p_assinatura_b64`: Representation of the spectral projection basis.
- `timestamp_operacao_str`: Temporal record of execution.

All fields are serialised deterministically.

4.3 Corruption detection

If the key indicates integrity, the audit merely records that the operation was executed.

If the key indicates corruption, `auditoria_verificado` reports the index of the block where divergence was detected.

4.4 Relationship with the key

The audit is mathematically consistent with the key.

Key and audit describe the same operation.

4.5 Immutability

Once generated, the audit is not modified.

Any alteration to any field invalidates the audit.

4.6 Exposure

The audit may be stored, copied, and transported freely.

Possession of the audit does not grant access to the file content.

5 Operational Properties

The CIP protocol is deterministic.

The same input produces the same output.

The protocol operates entirely offline.

It does not depend on servers, external authorities, or network infrastructure.

The protocol is reproducible.

Any party may execute the binary and obtain the same results.

The protocol has no central authority.

There is no privileged issuer.

Validity is established exclusively by mathematical coherence.

A minimal public experimentation environment is provided.

The protocol is executable and verifiable in an uncontrolled environment (Google Colab).

Operational notebook: https://colab.research.google.com/drive/1vmYMYuNgryfgKzC09yHnZX36Erj35i8U?usp=sharing

The CIP binary operates in an implacable and binary manner because it treats central symmetry as a structural condition of coherence.

6 Trust Model

The CIP protocol does not shift trust to third parties.

Trust is not placed in servers.

Trust is not placed in certification authorities.

Trust is placed exclusively in the reproducibility of the operator.

Whoever produces an artefact can verify that artefact.

Whoever receives an artefact can verify the artefact independently.

There is no mathematical distinction between verifier and verified.

7 Scope Limits

The CIP protocol does not provide anonymity.

The CIP protocol does not provide identity authentication.

The CIP protocol does not replace communication protocols.

The CIP protocol does not prevent file copying.

The CIP protocol only measures coherence, transforms, and verifies the persistence of coherence.

8 Minimal Governance

The CIP protocol requires operational maintenance.

This maintenance includes compilation, packaging, distribution, and binary versioning.

Governance does not interfere with the mathematical functioning of the operator.

The operator remains verifiable regardless of who distributes it.

Any party that masters the specification may assume compilation and distribution functions.

Part of any operational surplus, when present, should be directed towards strengthening basic mathematical education in structurally under-resourced regions, as a mechanism for preserving the very base that sustains the operator.

The allocation of any surplus to mathematical education should follow principles of global equity, allowing regions of high operational density to finance the intellectual base in regions of infrastructural vulnerability, as a mechanism for preserving the universality of the metric.

Governance associated with the protocol must be distributed, compatible with the absence of central authority of the operator, allowing multiple independent poles of compilation, distribution, and surplus allocation, all verifiable under the same public specification.

Surplus allocation occurs through voluntary and auditable pacts between independent entities, without a central mediating body, with the integrity of such pacts guaranteed by the same mechanisms of the operator.

Costs associated with operating the protocol do not represent payment for access, but an explicit commitment to preserving coherence. They constitute a cost of maintaining structural invariance, not monetisation of the operator.

The operator is free. Coherence is not.

This commitment is assumed exclusively by commercial uses that extract direct economic value from the operation, while remaining free for public, educational, and non-commercial purposes.

The official specification and public discussion channels of the protocol are available on a static page hosted in a GitHub repository. Permanent archival registration is maintained via Zenodo.

9 Closing

The CIP protocol is an engineered application of a known statistical structure.

It claims no mathematical novelty.

It proposes no physical interpretation.

It does not depend on unproven hypotheses.

The work presented merely describes an operator.

The operator has been specified.

The operator has been implemented.

The operator is reproducible.

The operator is verifiable.

The operator does not solicit adherence.

Nothing beyond this description.

Any social, political, or economic consequences arising from use of the operator lie outside the scope of this document.